

Week01

- tips and tricks
- operators
- control flow
- Cow.Java



operators

(except for bitwise operators, which we'll do later maybe)

assignment operator

assignment operator

- **= assigns** the value on the right-hand side to the variable on the left-hand side
 - `int i = 0; // 0 ("int i now has the value 0")`
 - `double foo = coolFunction();`
- 🤖 the assignment operator returns the value it assigned
this is usually pretty confusing
 - `boolean b = false;`
 - `boolean c = (b = foo());`

arithmetic operators

basic arithmetic (number) operators

- **+** **adds** two numbers
- **-** **subtracts** two numbers
- ***** **multiplies** two numbers
- **/** **divides** two numbers
 - 🤖 **an int divided by an int is an int**
 - `int foo = 8 / 2; // 4`
 - `int bar = 7 / 2; // 3`
 - Java "throws away the remainder"
- **-** returns the **negative** of a number
 - `int bar = -7; // -7 ("negative 7" or "minus 7")`
 - `int baz = -bar; // 7`

modulo

- `x % y` returns the **remainder** of (`x / y`) and is read "x modulo y"
 - `int foo = 17 % 5; // 2` ("17 divided by 5 is 3 remainder 2")
- 🤖 `%` probably doesn't do what you expect for negative numbers; if `x` can be negative, use `MODULO(x, y)` instead
 - `5 % 3 // 2`
 - `4 % 3 // 1`
 - `3 % 3 // 0`
 - `2 % 3 // 2`
 - `1 % 3 // 1`
 - `0 % 3 // 0`
 - `-1 % 3 // -1` 😞
 - `MODULO(-1, 3) // 2` 😊👍

modulo

$$7 \% 3$$

$$\begin{array}{r} 2 \text{ R } 1 \\ 3 \overline{) 7} \\ \underline{-6} \\ 1 \end{array}$$

$$\text{☕} \quad -7 \% 3$$

$$\begin{array}{r} -2 \text{ R } -1 \\ 3 \overline{) -7} \\ \underline{-6} \\ -1 \end{array}$$

$-7 - (-6) = -7 + 6 = -1$

$$\text{🐮} \cdot \text{☕} \quad \text{MODULO}(-7, 3)$$

$$-1 + 3 = 2$$

%

- `x % y` returns the **remainder** of (`x / y`) and is read "x modulo y"
 - `int foo = 17 % 5; // 2` ("17 divided by 5 is 3 remainder 2")
- 🤖 `%` probably doesn't do what you expect for negative numbers; if `x` can be negative, use `Math.floorMod(x, y)` instead
 - `5 % 3 // 2`
 - `4 % 3 // 1`
 - `3 % 3 // 0`
 - `2 % 3 // 2`
 - `1 % 3 // 1`
 - `0 % 3 // 0`
 - `-1 % 3 // -1` WAIT WHAT 😞
 - `MODULO(-1, 3) // 2` 😊👍

MODULO(x, y)

- `MODULO(5, 3) // 2`
- `MODULO(4, 3) // 1`
- `MODULO(3, 3) // 0`
- `MODULO(2, 3) // 2`
- `MODULO(1, 3) // 1`
- `MODULO(0, 3) // 0`
- `MODULO(-1, 3) // 2`
- `MODULO(-2, 3) // 1`
- `MODULO(-3, 3) // 0`
- `MODULO(-4, 3) // 2`
- `MODULO(-5, 3) // 1`
- `MODULO(-6, 3) // 0`
- `MODULO(-7, 3) // 2`

logical operators

logical operators (1/2)

- `||` returns whether the left-hand side **or** the right-hand side is true
 - `(true || true) // true`
 - `(true || false) // true`
 - `(false || true) // true`
 - `(false || false) // false`
- `&&` returns whether the left-hand side **and** the right-hand side are true
 - `(true && true) // true`
 - `(true && false) // false`
 - `(false && true) // false`
 - `(false && false) // false`
- `!` returns the opposite of a *boolean*, and is read as "not"
 - `(!true) // false ("not true")`
 - `(!false) // true`

logical operators (2/2)

```
- // example, step by step
- boolean a = (2 + 2 == 5); // false
- boolean b = true;        // true
- boolean c = (a || b);    // true
- boolean d = !c;          // false

- // same thing all on one line
- boolean d = !((2 + 2 == 5) || true); // false

- // equivalent code
- boolean d = false;
```

logical operator short-circuiting

- `(false && foo())` "lazily" evaluates to **false** without evaluating `foo()`
- `(true || foo())` "lazily" evaluates to **true** without evaluating `foo()`

comparison operators

equality (is equal to)

- `==` returns whether the left-hand side **is equal to** the right-hand side
 - `boolean b = (foo == bar);`
 - `if (foo == bar) { ... }`
-  this does **NOT** work for *String*'s
 - instead, use `(stringA.equals(stringB))`
-  this (usually) does **NOT** work for *double*'s
 - instead, use `ARE_EQUAL(double1, double2)`

is greater than, is less than

- > returns whether the left-hand side **is greater than** the right-hand side
- < returns whether the left-hand side **is less than** the right-hand side

convenient operators

(feel free to ignore these for now)

inequality

- != returns whether the left-hand side **is not equal to** the right-hand side
 - (left != right) is exactly the same as !(left == right)

greater than or equal to, less than or equal to

- >= returns whether the left-hand side **is greater than or equal to** the right-hand side
 - (left >= right) is basically the same as
((left > right) || (left == right))
greater-than or equal
- <= returns whether the left-hand side **is less than or equal to** the right-hand side

arithmetic assignment operators

- a += b; // a = a + b;
- a -= b; // a = a - b;
- a *= b; // a = a * b;
- a /= b; // a = a / b;

String concatenation

- + concatenates two `String`'s
 - ```
// String foo = "Hello".concat("World");
String foo = "Hello" + "World"; // "HelloWorld"
```
- it also does some conversions for you  
(very convenient, kind of confusing)
  - ```
// String foo = "Hello".concat(Integer.toString(2));  
String foo = "Hello" + 2; // "Hello2"
```

increment operator

- to **"increment"** means to increase the value of a number by one
 - `i = i + 1;`
 - `i += 1;`
- the **pre-increment** `++i` increments `i` and returns the new value of `i`
 - `j = ++i; // i = i + 1;`
 - `// j = i;`
- the **post-increment** `i++` increments `i` and returns the old value of `i`
 - `j = i++; // j = i;`
 - `// i = i + 1;`

decrement operator

- to **"decrement"** means to decrease the value of a number by one
 - `i = i - 1;`
 - `i -= 1;`
- the **pre-decrement** `--i` decrements `i` and returns the new value of `i`
 - `j = --i; // i = i - 1;`
 - `// j = i;`
- the **post-decrement** `i--` decrements `i` and returns the old value of `i`
 - `j = i--; // j = i;`
 - `// i = i - 1;`

flow control

the execution of a Java program
starts at the top of `main(...)`
and flows down down down

let's step through this program in our minds, and
then in a debugger

```
class Main extends Cow {  
    public static void main(String[] arguments) {  
        double a = 3.0;  
        double b = 4.0;  
  
        double result = Sqrt(a * a + b * b);  
        PRINT(result);  
    }  
}
```

functions (preview)

functions (preview)

- when a **function** is called, control flow jumps to the top of the function, flows down through it, and then jumps back to right after the function call

```
class Main {
    static double pythagoreanTheorem(double a, double b) {
        return Math.sqrt(a * a + b * b);
    }

    public static void main(String[] arguments) {
        double hypotenuse = pythagoreanTheorem(3.0, 4.0);
        System.out.println(hypotenuse);
    }
}
```

ASSERT

ASSERT(condition);

- an **assert statement** crashes the program if its condition is false
 - `ASSERT(false)`; crashes the program no matter what

```
class Main {
    static double pythagoreanTheorem(double a, double b) {
        ASSERT(a > 0.0);
        ASSERT(b > 0.0);
        return Math.sqrt(a * a + b * b);
    }

    public static void main(String[] arguments) {
        double hypotenuse = pythagoreanTheorem(3.0, 4.0);
        PRINT(hypotenuse);
    }
}
```

assert condition; (Java Java)

- 🐘 Java's asserts are actually disabled by default (wat)
 - you can enable them in Eclipse, but it's easy to forget to do

if and else

if (condition) { body }

- an **if statement** lets a program make a decision (instead of just stepping down down down forever down)

```
int choice = getIntFromUser();

if (choice == 0) {
    System.out.println("The user chose 0. What a fine choice");
}
```

`if (...) { if-body } else { else-body }`

- the body of an **else statement** is executed if its corresponding if statement's condition is false

```
int choice = getIntFromUser();  
  
if (choice == 0) {  
    System.out.println("The user chose 0. What a fine choice");  
} else {  
    System.out.println("The user did not choose 0.");  
    System.out.println("How avant-garde!");  
}
```

`if (...) { ... } else if (...) { ... } ... else { ... }`

- if and else statements can be chained together
 - this is great for decisions with many options

```
int choice = getIntFromUser();  
  
if (choice == 0) {  
    ...  
} else if (choice == 1) {  
    ...  
} else if (choice == 2) {  
    ...  
} else {  
    ASSERT(false);  
}
```

while and do...while

`while (condition) { body }`

- a **while loop** repeats a block of code as long as the condition holds (if the condition is false the first time we see it, we never execute the body)
 - `while (true) { ... }` is an infinite loop

```
while (!gameOver) {  
    ...  
  
    if (player.health <= 0) {  
        gameOver = true;  
    }  
}
```

`do { ... } while (...);`

- a **do...while** loop is (sometimes) useful when you know you need to do something at least once (and then potentially a bunch more times)

```
int choice;  
do {  
    choice = getIntFromUser();  
} while (!(0 <= choice && choice <= 2));
```

```
int choice = getIntFromUser();  
while (!(0 <= choice && choice <= 2)) {  
    choice = getIntFromUser();  
}
```

for

`for (...; condition; ...) { ... } (1/2)`

- a **for loop** can be a convenient alternative to a while loop

```
for (int i = 0; i < n; ++i) {  
    ...  
}
```

```
{  
    int i = 0;  
    while (i < n) {  
        ...  
        ++i;  
    }  
}
```

`for (...; condition; ...) { ... } (2/2)`

- for loops can be kind of wild (probably stick with simple ones for now)

```
for (double a = 10.0; a > 1.01; a = Sqrt(a)) {  
    ...  
}
```

```
for (int j = n - 1, i = 0; i < n; j = i++) {  
    ...  
}
```

```
for (;;) {  
    ...  
}
```

break and continue

break

- **break** breaks out of a loop

```
while (true) {  
    ...  
    if (player.health <= 0) {  
        break;  
    }  
}
```

continue

- **continue** continues to the next iteration of a loop

```
for (int i = 0; i < chairs.length; ++i) {  
    if (chairs[i].hasStudentSittingInIt) {  
        continue;  
    }  
    ...PRINT student's name...  
}
```

exceptions to the rule that
"scope is the same as
curly braces"

`for (...; ...; ...) { ... }`

- variables declared inside the parentheses of a **for loop** are not available outside of the for loop (this is probably the behavior you already expected)

Compile Error: cannot find symbol i

```
class Main {
    public static void main(String[] arguments) {
        for (int i = 0; i < 10; ++i) {
            ...
        }
        PRINT(i);
    }
}
```

☠ if, else, for, while without braces (1/2)

- ☠ if you (intentionally or unintentionally) forget your curly braces, then Java will assume you wanted them go around the first statement after the if (...), else, for (...), or while (...)
- in this class, i highly recommend always using curly braces

```
if (choice == 0)
    System.out.println("The user chose 0. What a fine choice");
```

```
if (choice == 0) {
    System.out.println("The user chose 0. What a fine choice");
}
```

☠ if, for, while without braces (2/2)

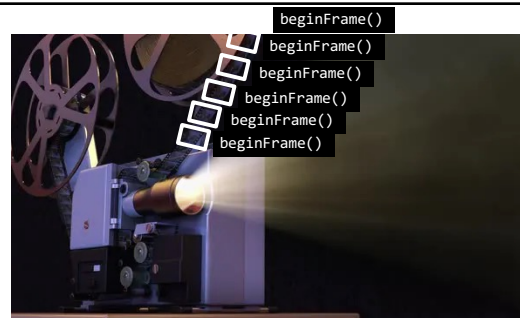
```
if (choice == 0)
    PRINT("The user chose 0. What a fine choice");
else
    PRINT("The user did not choose 0.");
    PRINT("How avant-garde!");
```

```
if (choice == 0) {
    PRINT("The user chose 0. What a fine choice");
} else {
    PRINT("The user did not choose 0.");
} // whoops!
    PRINT("How avant-garde!");
```

Cow.Java

overview

during a **frame** (like a frame of a movie)
we **update** and **draw** the world



[discussion of update-draw loop]

your best friend

